

TEORI BAHASA DAN AUTOMATA

Modul Pembelajaran: Non-Deterministic Finite Automata (NFA)

Penjelasan mendalam mengenai konsep non-determinisme, transisi epsilon (ϵ), dan berbagai studi kasus perancangan mesin NFA dari dasar hingga mahir.

Contents

1	Mengenal Non-Deterministic Finite Automata (NFA)	2
1.1	Apa itu Mesin NFA dan Bedanya dengan DFA?	2
1.2	Syarat Diterimanya String di NFA	2
2	Transisi Kosong: Epsilon (ϵ)	3
2.1	Gimana Cara Kerja Transisi Epsilon?	3
3	Definisi Formal dan Tabel Transisi NFA	4
3.1	Tabel Transisi NFA	4
4	Kumpulan Studi Kasus Perancangan NFA	5
4.1	Kasus 1: Pola Berakhiran "01"	5
4.2	Kasus 2: Mengandung Substring "11" atau "00"	5
4.3	Kasus 3: NFA Tanpa Jalan Buntu (Mirip DFA)	5

1 Mengenal Non-Deterministic Finite Automata (NFA)

1.1 Kenapa Harus Ada NFA?

Di materi sebelumnya, kita udah bahas DFA yang sifatnya kaku dan serba pasti. Kalau di DFA, tiap *state* wajib punya satu jalan keluar untuk setiap simbol, nggak boleh lebih dan nggak boleh kurang.

Tapi pada praktiknya, pas kita mau mendeteksi pola bahasa yang agak rumit, ngerancang DFA itu bisa bikin pusing karena *state*-nya jadi meledak (banyak banget). Nah, di sinilah *Non-Deterministic Finite Automaton* (NFA) jadi pahlawan. NFA ini ngasih kita kebebasan penuh buat ngerancang mesin dengan cara yang lebih fleksibel dan sesuai sama intuisi manusia.

Di NFA, kalau mesin dapet satu input, dia bisa bereaksi dengan tiga cara ini:

- **Multiple Paths (Bercabang):** Satu karakter bisa bikin mesin loncat ke lebih dari satu *state* sekaligus. Mesin bakal seolah-olah "ngekloning" dirinya sendiri buat nyusurin semua jalan itu.
- **Dead End (Jalan Buntu):** Kalau mesin dapet karakter yang nggak ada panah transisinya, kloningan mesin di jalur itu langsung mati atau gagal.
- **Epsilon Transition (ϵ):** Ini kemampuan buat loncat atau pindah ke *state* lain secara gaib, tanpa perlu ngebaca karakter apa pun dari pita input!

1.2 Aturan Main Diterimanya String di NFA

Karena mesin NFA bisa bercabang jadi banyak kloningan pas lagi ngebaca string, syarat biar string itu **diterima** (*Accept*) simpel banget: **Asalkan ada minimal SATU aja jalur kloningan yang sukses berhenti di Final State pas string-nya habis dibaca, maka string itu dianggap sah dan diterima.**

Nggak peduli kalau ada puluhan kloningan lain yang mati di jalan buntu atau nyasar ke *state* biasa. Kalau semuanya gagal, baru deh string itu dinyatakan **ditolak** (*Reject*).

2 Kumpulan Studi Kasus Perancangan NFA

Biar makin paham bedanya NFA dan DFA, mari kita bedah berbagai macam studi kasus. Merancang NFA itu rasanya jauh lebih gampang, karena kita tinggal bikin jalur khusus buat pola yang benar aja, sisanya biarin buntu!

2.1 Kasus 1: Pola Berakhiran "01"

Kita mau bikin mesin yang terima string biner, asalkan dua huruf terakhirnya adalah "01". Depannya mau angka apa aja, sepanjang apa pun, bebas. Alfabetnya $\Sigma = \{0, 1\}$.

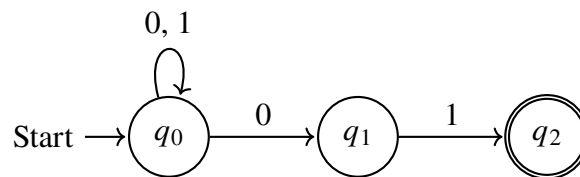


Figure 1: NFA String Berakhiran "01"

Kenapa NFA ini gampang banget dibaca?

- Di state q_0 , kalau dia dapet angka '0', mesin ini ****nebak**** atau ****bercabang****. Satu kloningan muter tetep di q_0 (berasumsi ini bukan '0' yang terakhir), dan satu kloningan lagi maju ke q_1 (berasumsi ini adalah awal dari akhiran "01").
- Kalau asumsinya salah (misal abis '0' malah '0' lagi), kloningan yang ada di q_1 bakal ****mati**** karena nggak ada panah '0' dari q_1 . Tapi kloningan yang di q_0 masih hidup dan siap nebak lagi.

2.2 Kasus 2: Mengandung Substring "010"

Sekarang kita cari string yang di tengah-tengahnya (atau di mana pun) punya pola berurutan "010". Kalau udah ketemu "010", belakangnya mau diisi apa aja bebas.

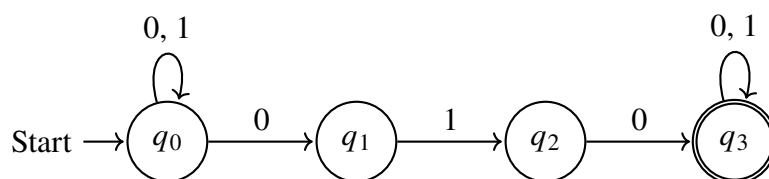


Figure 2: NFA Mendeteksi Substring "010"

Alur Logika: State q_0 tugasnya cuma nunggu. Tiap kali ada angka '0' lewat, dia ngutus satu kloningan buat masuk ke terowongan $q_1 \rightarrow q_2 \rightarrow q_3$. Kalau di tengah jalan urutannya rusak (misal abis '0' malah '0' lagi di q_1), kloningan itu bakal gugur. Tapi kalau berhasil lolos sampai q_3 , dia bakal santai nge-loop di sana sampai string habis.

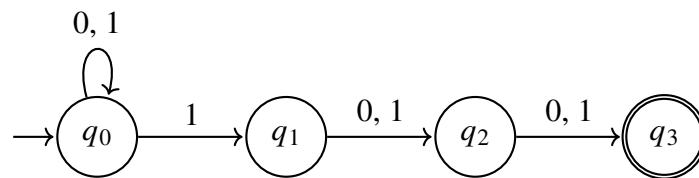


Figure 3: NFA: Karakter ketiga dari belakang adalah 1

2.3 Kasus 3: Karakter Ketiga dari Belakang adalah "1"

Ini adalah contoh klasik di mana NFA jauh menang telak dibanding DFA. Kita mau terima string yang kalau dihitung dari kanan ke kiri, karakter ketiganya pasti angka '1'. Contoh yang diterima: "00100", "111", "0101101".

Rahasia Kekuatan NFA: Bayangin kalau kita bikin ini pakai DFA, kita butuh *state* buat nginget semua kombinasi 3 huruf terakhir (kayak 000, 001, 010, dst), total butuh 8 *state* yang ribet panahnya. Tapi pakai NFA, kita cuma butuh 4 *state*! Tiap ketemu angka '1', q_0 ngutus kloningan maju ke q_1 . Kloningan ini dikasih jatah persis dua langkah lagi ke q_3 . Kalau stringnya keburu habis sebelum nyampe q_3 , atau stringnya masih sisa pas dia udah di q_3 (mati, karena q_3 nggak punya panah keluar), berarti itu bukan '1' yang ketiga dari belakang. Cerdas banget, kan?

2.4 Kasus 4: Menggunakan Jalan Tol Epsilon (ϵ)

Kita mau deteksi string yang punya "11" berurutan **ATAU** "00" berurutan. Ini gampang banget diselesaikan pakai *Epsilon Transition* (loncat gaib tanpa baca karakter).

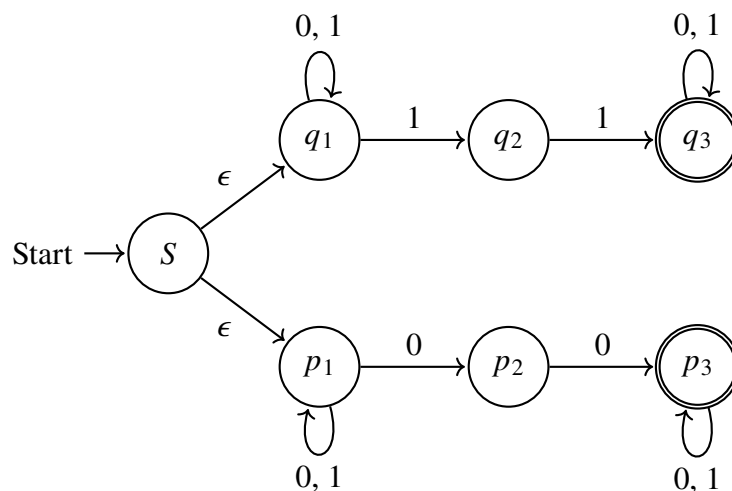


Figure 4: NFA Epsilon untuk mendeteksi substring "11" atau "00"

Pas mesin dinyalain, berkat transisi ϵ , mesin seketika itu juga terbelah jadi dua tanpa perlu baca input apa pun. Satu kloningan *standby* di q_1 tugasnya nyari "11". Kloningan satunya lagi *standby* di p_1 tugasnya nyari "00". Mereka kerja bareng secara paralel. Siapa yang berhasil nyampe garis *finish* duluan (q_3 atau p_3), dialah yang bikin stringnya diterima.

2.5 Kasus 5: Pengecekan Pola Terstruktur $(a \cup b)^*abb$

Kita mau konversi ekspresi reguler $(a \cup b)^*abb$ ke dalam NFA. Artinya, depannya boleh diisi kombinasi huruf 'a' atau 'b' sebanyak apa pun, asalkan ditutup dengan tepat pola "abb".

Alfabetnya $\Sigma = \{a, b\}$.

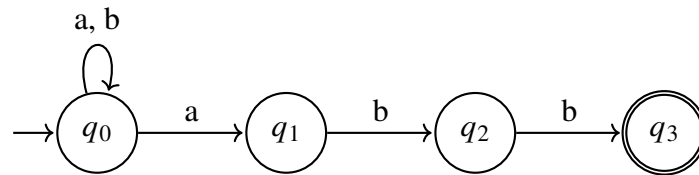


Figure 5: NFA untuk pola regex $(a \cup b)^*abb$

Simulasi kalau kita kasih input "ababb":

- Baca 'a' ke-1: Mesin bercabang. Ada yang stay di q_0 , ada yang maju ke q_1 .
- Baca 'b' ke-2: Yang di q_0 tetep di q_0 . Yang di q_1 maju ke q_2 .
- Baca 'a' ke-3: Yang di q_0 bercabang lagi ke q_0 dan q_1 . Yang di q_2 mati (karena dari q_2 butuhnya 'b').
- Baca 'b' ke-4: Kloningan dari q_0 tetep di q_0 . Kloningan dari q_1 maju ke q_2 .
- Baca 'b' ke-5: Kloningan dari q_0 tetep di q_0 . Kloningan dari q_2 maju ke q_3 .
- String habis! Karena ada satu kloningan yang sukses nangkring di q_3 (Final State), string "ababb" **Diterima**.

2.6 Kasus 6: Bahasa Tanpa Salah Satu Huruf (Missing Letter)

Misalkan kita punya alfabet dengan 3 huruf $\Sigma = \{a, b, c\}$. Kita mau mesin nerima string asalkan ada **salah satu huruf yang nggak pernah muncul** sama sekali. Pilihannya: String nggak punya 'a', ATAU nggak punya 'b', ATAU nggak punya 'c'.

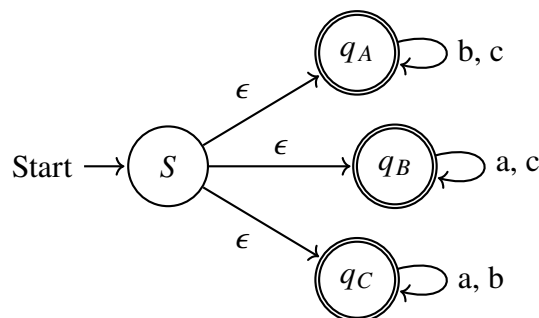


Figure 6: NFA String Kehilangan Minimal Satu Huruf Alfabet

Alur Logikanya: Begitu jalan, ϵ -transition bikin mesin pecah jadi 3 divisi pemeriksa: 1. Divisi q_A tugasnya nampung string yang **nggak ada 'a'**. Makanya dia cuma mau nerima huruf 'b' dan 'c'. Kalau ada 'a' nyasar masuk, divisi ini langsung gagal/mati. 2. Divisi q_B khusus nampung string yang **nggak ada 'b'** (cuma nerima 'a' dan 'c'). 3. Divisi q_C khusus nampung string yang **nggak ada 'c'** (cuma nerima 'a' dan 'b').

Kalau kita kasih input "bcbbcb", divisi q_B dan q_C bakal rontok di tengah jalan. Tapi divisi q_A bakal bertahan muter-muter terus dan sukses. Karena minimal ada satu divisi yang *survive* sampai akhir, string "bcbbcb" dinyatakan sah dan **diterima**!